01 / 핵심 결과

symbol_index와 계층 요약 RAPTOR, 어떤 방식이 얼마나 나은가

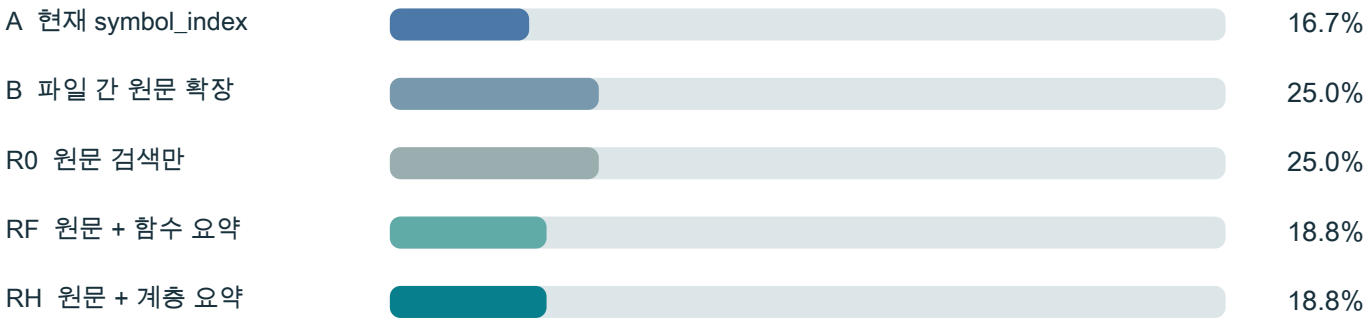
현재 구현과 확장 구현을 같은 문제·모델·근거 자료 예산에서 직접 비교한 재현 가능한 파일럿

결론부터

서로 다른 48문제 중 현재 방식은 8개, 계층 요약 방식은 9개를 맞혔습니다.

현재 방식 A	계층 요약 RH	정답률 차이 RH - A
16.7%	18.8%	+2.1%p
24 / 144 응답 정답	27 / 144 응답 정답	고유 문제 48개 · 프로젝트 6개

다섯 가지 방식의 정답률



RH - A의 프로젝트 단위 탐색적 95% 구간: -8.3~+12.5%p. 6개 프로젝트의 효과 추정치이며, 통계적으로 우위가 확정됐다는 뜻은 아닙니다.

요약 자체의 기여를 분리

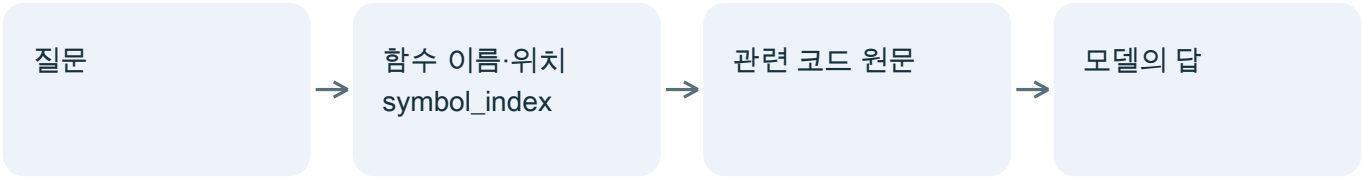
같은 원문 검색에 계층 요약을 더하자 주 정답률이 6.2%p 낮아졌습니다. 출력 순서 오류만 제외한 사후 보조 점수는 두 방식 모두 25.0%입니다. 현재 방식과의 차이에는 검색 방식과 파일 간 원문 확장도 함께 작용합니다.

해석 범위

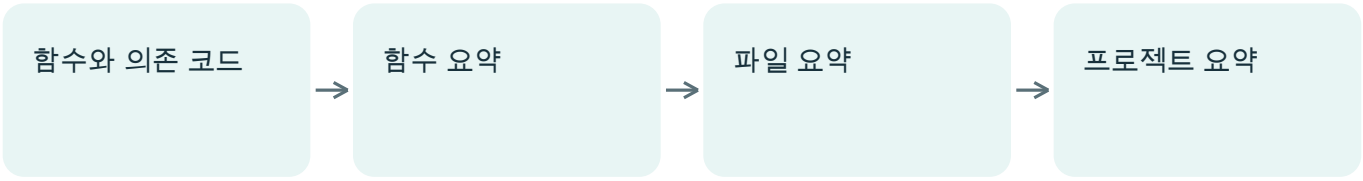
알고리즘이 다른 작은 합성 Python 프로젝트 6개의 코드 이해 평가입니다. 공통 파일 구조·보조 모듈을 공유합니다. 실제 저장소·코드 수정·요약 갱신 및 원논문 RAPTOR 전체는 평가하지 않았습니다.

02 / 무엇을 비교했는가

이름을 찾는 색인에서, 원문과 여러 수준의 요약을 함께 찾는 방식으로



A: 후보 24개 중 모델이 심볼을 고르고 같은 파일의 원문을 묶습니다.



RH: 함수 묶음 → 파일 → 프로젝트 순서로 요약을 만들고, 원문과 모든 수준의 요약을 함께 검색합니다.

구분	방법	차이
A	현재 filtered_select_bundle	모델이 심볼 선택 + 같은 파일 확장
B	A의 파일 간 확장 대조군	같은 선택·2회 확장·최대 함수 8개
R0	요약 없는 검색 대조군	개별 원문 + 의존 묶음의 TF-IDF
RF	평면 요약	R0 + 함수 수준 요약
RH	계층 요약	RF + 파일 요약 + 프로젝트 요약

공정성을 위한 고정 조건

R0·RF·RH는 같은 원문 기반 단어 가중치·코사인 검색, 동일 원문 후보와 예산을 사용합니다. 개별 원문과 깊이 3 의존 묶음 중 이미 선택된 source URI로 완전히 덮이는 원문 노드만 건너뜁니다. 일부 중복은 가능합니다.

RAPTOR라는 이름의 범위

원논문의 임베딩·차원 축소·확률적 군집화 대신 AST·파일별로 묶고 TF-IDF로 찾습니다. 코드용 변형의 결과이며 원논문 전체 구현의 성능으로 볼 수 없습니다. B의 import 표기도 A와는 다릅니다.

03 / 실험 설계

정답은 실행으로 확인하고, 문제와 정답은 검색 자료에서 분리했습니다

데이터와 실행 조건	설계
개발 / 평가·언어	개발 1프로젝트 8문제 / 평가 6프로젝트 48문제. 질문은 영어
반복과 비교	문제당 3회 × 5방식, 총 720개 답변
주 실험 근거 자료 예산	1,024 모델 토큰; 질문·지시문 별도. 온전한 노드만 넣음
보조 실험	512토큰에서 R0·RH 각 1회; 예산 민감도만 확인
모델 / 장비	Qwen2.5-Coder-7B-Instruct Q5_K_M, 로컬 llama.cpp Apple M3 Pro · 36GiB
생성 설정	temperature 0, seed 1, context 8192, 답변 최대 128토큰
실행 순서	고정 난수로 문제·방식·반복 순서를 섞음; prompt cache 비활성
채점	주지표: 정답 내용과 요청한 출력 순서를 모두 지킨 비율. 바깥 코드펜스만 제거

프로젝트마다 같은 분포로 8문제

유형	평가 문제	확인하는 능력
함수 하나 / 경계 조건	6 + 6	단일 함수와 임계값 처리
파일 간 흐름 / 자연어	12 + 12	의존 코드 연결과 함수명이 없는 질문
변경 영향 / 호출 구조	6 + 6	지정 입력의 영향과 직접 호출 목록

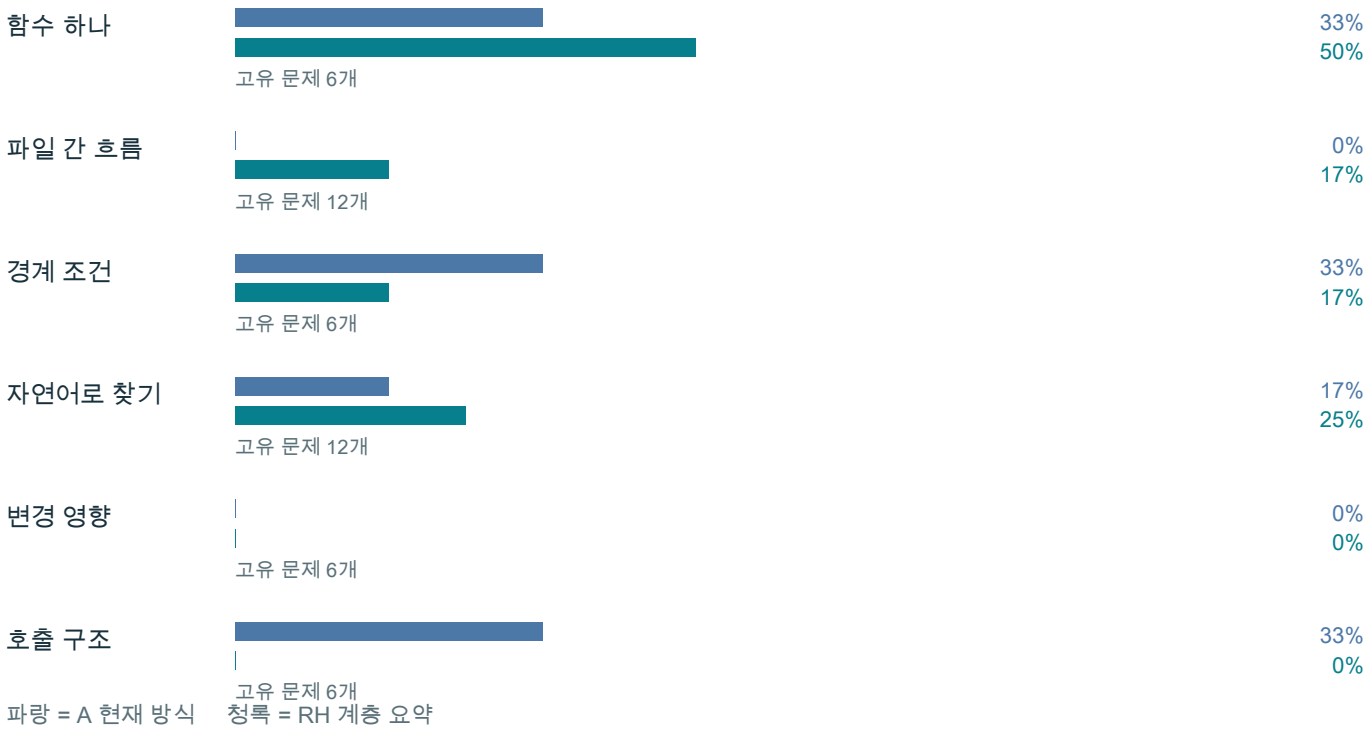
정답 검증

실행형 문제는 별도 Python 프로세스의 실제 반환값을 수작업 기대값과 대조했습니다. 변경 영향은 지정 입력의 변경 전후 실행, 구조 문제는 AST의 직접 호출 목록으로 확인했습니다.

요약에는 코드만 입력하고 질문·정답은 검색 폴더 밖에 두었습니다. 개발 단계에서 질문의 중복 JSON 지시를 제거하고 바깥 코드펜스만 허용하도록 고정한 뒤 평가했습니다. required_uris는 보수적인 참조 목록이며 최소 필요 근거는 아닙니다.

04 / 유형별 결과

어느 종류의 질문에서 차이가 생겼는가



방식	주 정답률	순서만 오류	순서무시 보조 정답률	지정 원문 회수율	모든 지정 원문 포함
A	16.7%	0회	16.7%	34.6%	20.8%
B	25.0%	3회	27.1%	76.6%	56.2%
R0	25.0%	0회	25.0%	100.0%	100.0%
RF	18.8%	9회	25.0%	59.4%	52.1%
RH	18.8%	9회	25.0%	56.9%	50.0%

정답률과 원문 회수율은 다릅니다

정답률은 실제 답이 맞은 비율입니다. 지정 참조 원문 회수율은 검증용 심볼 중 원문이 문맥에 실린 비율입니다. 출처 URI만 있으면 원문으로 세지 않았습니다. 지정 목록은 보수적이므로 회수율을 필요·충분한 사실의 비율로 해석하지 않습니다.

순서무시 보조 정답률은 같은 문자열 목록에서 정렬만 다른 답을 인정한 사후 진단입니다. 평가 도중 발견해 추가했으며 주지표를 대체하지 않습니다. 유형별 6~12문제의 차이도 탐색적으로만 해석합니다.

05 / 원인 분리

개선이 보인다면, 검색·파일 확장·요약 중 무엇의 효과인가

비교	정답률 차이와 95% 구간	분리해 보는 요인
RH - A	+2.1%p [95% 구간 -8.3, +12.5]	현재 방식에서 전체 변경의 효과
B - A	+8.3%p [95% 구간 +2.1, +16.7]	같은 선택기에서 파일 간 원문 확장
RH - R0	-6.2%p [95% 구간 -16.7, +4.2]	같은 검색기에 계층 요약 추가
RH - RF	+0.0%p [95% 구간 -6.2, +6.2]	함수 요약 위에 상위 요약 추가

문제별로 짝지어 보면

비교	새 방식 우세	기존 방식 우세	동률
RH-A	4	3	41
B-A	5	1	42
RH-R0	2	5	41
RH-RF	1	1	46

승패는 같은 문제의 3회 평균 정답 점수를 비교합니다. 144회 응답을 서로 독립인 144문제로 세지 않습니다. 모든 표의 차이는 뒤 방식 대비 앞 방식이며, 양수면 앞 방식이 우세합니다.

%p와 %는 구분합니다

A 16.7%에서 RH 18.8%로 바뀌면 정답률 차이는 +2.1%p입니다. 상대 변화는 +12.5%입니다. 본 보고서는 차이를 %p로 우선 표시합니다.

해석의 중심

RH - A만으로 요약의 독립적인 효과를 주장할 수 없습니다. 같은 검색기에 요약을 추가한 RH - R0와 상위 요약만 추가한 RH - RF를 함께 판단합니다.

06 / 속도와 비용

답을 얻는 비용과,
요약을 미리 만드는 비용을 나누어 봅니다

방식	평균 근거	평균 입력	평균 출력	평균 지연	중앙 / 95백분위
A	93	401	26	2.35초	2.21 / 3.45초
B	351	659	28	3.17초	3.36 / 4.51초
R0	1,016	1,163	18	4.26초	4.14 / 4.93초
RF	1,015	1,161	19	4.30초	4.15 / 5.11초
RH	1,014	1,160	17	4.22초	4.08 / 5.17초

입력토큰은 선택기+답변, 근거토큰은 전달된 코드·요약입니다. 1024는 상한입니다. 시간은 성공·실패의 3회 중앙값을 평균해 빠른 오답도 포함합니다. A·B의 공유 선택비용은 각각 전액 합산하며 토큰화·HTTP도 포함합니다. 둘 다 항상 정답인 5문제에서 RH/A 시간 비는 2.03배(1 미만이면 RH가 빠름)입니다.

평가 프로젝트 6개 전체의 최초 요약 구축

항목	측정값
요약 생성 호출	121회
요약 입력 / 출력 토큰	36,011 / 10,536
원문 준비·요약 구축 경과시간 합	569.5초 (9.5분)

시간 기준 손익분기점

RH의 평균 온라인 질의 시간이 A보다 짧지 않았습니다. 따라서 측정된 시간만으로는 요약 구축비를 회수하는 질의 수가 존재하지 않습니다. R0 대비 차이는 질의당 0.035초로 작습니다. 이를 그대로 외삽한 회수점은 프로젝트당 약 2,704회이며, 실제 회수를 검증한 결과가 아닙니다.

프로젝트당 원문 준비: A 0.867ms, 검색용 2.070ms. A와의 시간 차이에는 선택기 호출 제거와 입력량 차이가 함께 반영됩니다. 별도 임베딩 모델은 사용하지 않았고, 서버 시작·요약 갱신·전력비는 측정하지 않았습니다. API 요금으로 환산하지 않았습니다.

07 / 답이 달라진 실제 사례

같은 질문에서도
맞는 답과 틀린 답이 갈렸습니다

사례 1 | A 오답 → RH 정답: 상수의 이름과 실제 값

eval_authorization__own_write · 2번째 반복 (저장값 repeat=1)

자신의 문서에 쓰기 권한이 있을 때 authorize가 반환하는 객체

정답: {"allowed":true,"reason":"allowed"}
A (오답): {"allowed":true,"reason":"ALLOW_REASON"}
RH (정답): {"allowed":true,"reason":"allowed"}
R0 (정답): {"allowed":true,"reason":"allowed"}

A 문맥에는 ALLOW_REASON이라는 이름만 있고 값 정의가 없습니다. RH에는 원문의 ALLOW_REASON = "allowed"와 파일 간 보조 함수가 있습니다. 요약 없는 R0도 정답이므로, 이 개선을 요약 자체의 효과로 귀속할 수 없습니다.

사례 2 | R0 정답 → RH 오답: 정확한 직접 호출 관계

eval_pricing__direct_calls · 2번째 반복 (저장값 repeat=1)

checkout_total이 직접 호출하는 프로젝트 함수 세 개

정답: ["line_subtotal","member_discount","sales_tax"]
R0 (정답): ["line_subtotal","member_discount","sales_tax"]
RH (오답): ["checkout_total"]

R0는 실제 함수 본문을 읽고 세 함수를 맞혔습니다. RH는 1024토큰을 모두 요약으로 채웠고 원문 노드는 0개였습니다. 함수 이름 일부는 요약에 있지만 정확한 직접 호출 관계가 약해졌습니다. 이 변화가 오답의 유일한 원인이라고 확정한 것은 아닙니다.

사례를 읽는 방법

같은 문제·같은 반복의 답과 실제 문맥을 맞춰 검토한 사후 사례입니다. 상수의 원문 접근, 검색 순위, 요약 내용, 모델의 읽기 능력이 함께 달라질 수 있습니다. 전체 우열은 앞의 집계 결과로 판단해야 합니다.

근거: evidence/contrastive_cases.json. 답변 ID와 문맥 SHA-256을 최종 results_eval_main.jsonl에 대조해 확인합니다.

08 / 실패와 예외

평균 점수에 가려진 사례를 원문 기록으로 확인합니다

사례 1 | 안전재고: 빠진 상수와 계산 실패

eval_inventory__safety_buffer · 3번째 반복 (저장값 repeat=2)

재고 8개, 기예약 5개에서 1개 요청. 현재 시각은 200입니다.

정답: {"expires":200,"status":"denied","units":0}
R0: {"status":"partial","units":3,"expires":1090}
RH: {"status":"partial","units":1,"expires":1080}

8 - 5 - 안전재고 3 = 0이므로 예약은 거절됩니다. R0는 지정 원문을 모두 읽고도 오답입니다. RH 문맥에는 안전재고 값 3이 없어 정확한 계산 근거가 빠졌습니다. 이 누락이 특정 오답의 유일한 원인이라고 확정한 것은 아닙니다.

사례 2 | 환불: 올바른 공식을 보고도 틀린 답

eval_pricing__refund · 3번째 반복 (저장값 repeat=2)

1001센트를 지불한 3개 중 2개를 환불하는 계산입니다.

정답: 667
R0: 1001
RH: 1001

1001 × min(2, 3) // 3 = 667입니다. 두 조건 모두 문맥에 원문 공식이 있고 RH 요약도 정확하지만 둘 다 1001이라고 답했습니다. 이 실패는 검색 누락만으로 설명할 수 없습니다. 모델 내부의 계산 과정까지 관찰한 것은 아닙니다.

전체 방식의 실패 분포

값 오답 525회 · JSON 오류 27회 · 답변 보류(null) 15회 · 형식 오류 3회

별도 요약 관찰 · 채점되지 않은 사례

한 요약은 원문에 없는 URL 인코딩 동작을 덧붙였습니다. 관련 문자는 평가 질문에서 시험하지 않았으므로, 이를 측정된 오답의 원인이나 전체 요약 오류율로 계산하지 않았습니다.

두 사례는 설명을 위해 선택한 동반 실패이며 RH의 승패나 대표적인 오류 빈도를 뜻하지 않습니다. 근거: evidence/qualitative_audit.json 및 해당 결과의 실제 context.

09 / 불확실성과 판단

이번 결과로 말할 수 있는 것과, 다음 실험이 필요한 것을 구분합니다

권고 · 5개 방식의 후속 검증 우선순위

이번 조건에서는 RH의 정답률이 높고 A가 빨랐습니다. 최고 정답률도 25.0%인 파일럿입니다. 요약 없이 파일 간 원문을 잇는 B도 25.0%였고, 3.17초로 같은 점수인 R0의 4.26초보다 빨랐습니다. B를 실제 저장소에서 먼저 검증합니다.

확인한 범위	아직 확인하지 않은 범위
합성 Python 6개: 각 5파일·31~32원문 노드	실제 대규모 저장소, 언어·프레임워크 다양성
48문제의 정확한 반환값·호출 관계	버그 수정 성공률, 개발자의 실제 작업 시간
고정 모델·프롬프트·근거 자료 예산	다른 모델·임베딩·요약 전략과의 일반화
한 번 생성해 고정한 요약과 최초 준비 비용	요약 생성 변동성, 오래된 요약, 부분 갱신 비용
AST/from-import로 확인 가능한 의존관계	동적 디스패치·상속·복잡한 상대 import

불확실성을 계산한 방법

문제별 반복 평균을 만들고 프로젝트 단위 복원 추출의 모든 조합(6^6 = 46,656개)을 계산해 2.5·97.5백분위수를 표시했습니다. 6개 표본의 탐색적 구간입니다. 정확 부호 전환 검정의 최소 양측 p값은 0.03125이며, 2개 주요 비교의 Holm 보정 후 5% 유의 수준에서 우위 확정은 불가능합니다. 실제 보정 p값은 RH-A 1.0000, RH-R0 1.0000입니다.

3회 답이 같았던 조합: 240/240개. 반복은 독립 표본을 늘리지 않습니다. 공통 파일 구조·보조 모듈을 공유하므로 실제 저장소 6개의 다양성과 다릅니다. 후속 후보는 정답률, 동률이면 질의시간과 준비비 순으로 선택했습니다.

후속 검증의 우선순위

실제 저장소 확대와 사전 고정 문제 → 코드 수정·오래된 요약·갱신 비용 측정 → 임베딩·군집화를 적용한 RAPTOR와 원문 검색의 재비교 순으로 진행합니다.

512토큰 보조 실험: R0 16.7%, RH 8.3%, 차이 -8.3%p. 각 문제 1회만 수행한 민감도 점검으로 주 실험의 독립적인 재현으로 보지 않습니다.

10 / 재현과 근거

보고서의 숫자는 함께 저장한 원본 기록에서 다시 계산할 수 있습니다

파일 / 폴더	확인할 수 있는 내용
evidence/protocol.json	근거 자료 예산·모델 설정·대조군·주 비교 정의
evidence/dataset/	소스 코드, 평가 문제, 검증된 정답, 표본 구성
evidence/assets/	원문 묶음, 함수·파일·프로젝트 요약, 출처 연결
evidence/results_eval_main.jsonl	문제·방식·반복별 답변과 문맥, 비용, 성공 여부
evidence/results_eval_budget512.jsonl	512토큰 민감도 실험
evidence/analysis.json · per_question_results.csv	집계 수치와 스프레드시트로 볼 수 있는 문제별 결과
evidence/model_calls.jsonl	선택·요약·답변 모델의 프롬프트와 출력
evidence/environment.json	모델·구현 파일 해시, 시스템·실행기 버전
2nd/experiments/symbol_raptor/	데이터 생성, 실험 실행, 분석 및 보고서 생성 코드

실행 재현

동일 GGUF 모델·로컬 서버를 준비하고 experiment.py의 prepare → build → dev → eval → sensitivity를 실행합니다. 모델·포트는 스크립트를 참조합니다. 분석과 보고서는 저장된 결과만 읽습니다.

식별 정보	값 (전체 해시는 environment.json)
모델 SHA-256	586844eac4d6d6321689f019...
실험 코드 SHA-256	b8f664facdff58c7b410f704...
코퍼스 코드 SHA-256	719f9d68b892a95c2b7c0d50...
실행 환경	macOS-27.0.1-arm64-arm-64bit-Mach-O
llama.cpp	environment.json 참조

참고 자료

[1] Sarthi et al., RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval (2024). https://arxiv.org/html/2401.18059v1
[2] 기존 프로젝트 설계 문서: 2nd/Documents/Md_Files/2026-10-05_세_가지_포인터_방식의_정량평가_실험설계와_동작원리.md
[3] 본 보고서 근거: test_results/evidence/ 아래 실측 기록. 원논문 수치는 본 실험 결과로 재사용하지 않았습니다.